

Planificación de Procesos

Módulo 5

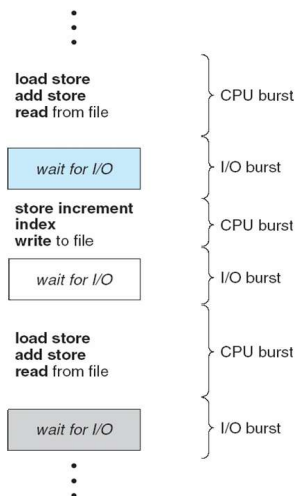
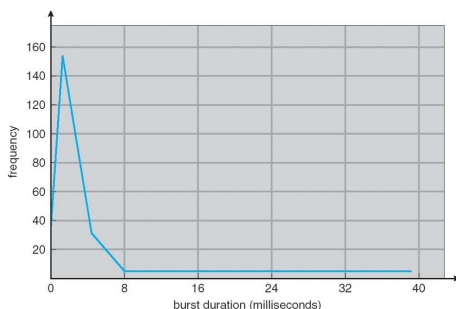
Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia "San Juan Bosco"

Planificación de Procesos

- Conceptos Básicos
- Criterios de Planificación
- Algoritmos de Planificación
- Planificación de Múltiples Procesadores
- Planificación en Tiempo Real
- Planificación de hilos
- Evaluación Algoritmos

Conceptos Básicos

- El objetivo de la multiprogramación es maximizar la utilización de la CPU.
- La ejecución de un proceso consiste de **ciclos** de ejecución de CPU y esperas de E/S.
- Distribución de ráfagas (burst) de CPU



JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

Planificador de CPU

- Selecciona un proceso, entre los que están en memoria, listos para ejecutar, y asigna la CPU.
- La decisión de planificar la CPU puede tener lugar cuando un proceso:
 1. Conmuta de **ejecutando** a estado de **espera**.
 2. Conmuta de **ejecutando** a estado de **listo**.
 3. Conmuta de **espera** a **listo**.
 4. **Termina**.
- La planificación de 1 y 4 es **no apropiativa**.
- Las otras planificaciones son **apropiativas**.

JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

Despachador

- El módulo **despachador**, es quien pasa el control de la CPU al proceso que fue seleccionado por el planificador de corto término; esto implica:
 - cambio de contexto
 - conmutación a modo usuario
 - salta a la dirección apropiada en el programa de usuario para reiniciar el proceso entrante.
- La **Latencia de despacho** es el tiempo que toma al despachador para detener un proceso e iniciar la ejecución de otro.

Criterios de Planificación

Para comparar los distintos algoritmos de planificación se proponen criterios:

- **Utilización de CPU** – Se pretende mantener la CPU tan ocupada como sea posible. $(\Sigma \text{Rafagas} / \text{procesamiento total}) * 100$
- **Procesamiento total (Throughput)** – número de procesos que completan sus ejecución por unidad de tiempo. (xej 10 procesos por segundo.)
- **Tiempo de retorno** –tiempo que tarda un proceso en ejecutarse. (desde que ingresa hasta que finaliza)
- **Tiempo de Espera** – tiempo que un proceso ha estado esperando en las distintas colas.
- **Tiempo de respuesta** – tiempo que transcurre desde que fue hecho un requerimiento hasta que se produce la primer respuesta, **no salida**.

Criterios de Optimización

- Maximizar la utilización de CPU
- Maximizar el procesamiento total
- Minimizar el tiempo de retorno
- Minimizar el tiempo de espera
- Minimizar el tiempo de respuesta

JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

- JRA © - LRS 2025

Planificación Primero-Entrar, Primero-Servido (FCFS)

Servicio por orden de llegada.

- Ejemplo:

Proceso	Tiempo de Ráfaga
P_1	24
P_2	3
P_3	3
- Suponer que los procesos llegan en el orden: P_1, P_2, P_3
La Diagrama de Gantt para la planificación es:

A horizontal timeline diagram representing the execution of three processes under FCFS scheduling. The timeline starts at 0 and ends at 30. Process P₁ (red) runs from time 0 to 24. Process P₂ (red) runs from time 24 to 27. Process P₃ (red) runs from time 27 to 30. The time values 0, 24, 27, and 30 are written below the timeline in green.

- Tiempo de espera para $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Tiempo medio de espera: $(0 + 24 + 27)/3 = 17$

JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

JRA © - LRS 2025 Sistemas Operativos –Planificación de Procesos

- JRA © - LRS 2025 Sistemas Operativos –Planificación de Procesos

JRA © - LRS 2025 Sistemas Operativos –Planificación de Procesos

JRA © - LRS 2025 Sistemas Operativos –Planificación de Procesos



- JRA © - LRS 2025 Sistemas Operativos –Planificación de Procesos

Planificación FCFS (Cont.)

Suponer que los procesos llegan en el orden

$$P_2, P_3, P_1.$$

■ La Diagrama de Gantt para la planificación es:



- Tiempo de espera para $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Tiempo medio de espera: $(6 + 0 + 3)/3 = 3$
- Mucho mejor que el caso anterior.
- *Efecto Convoy* → Muchos procesos cortos delante de un proceso largos

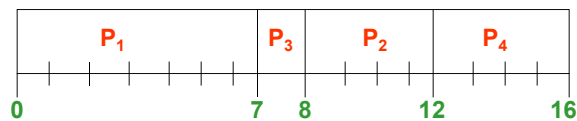
Planificación Job-Mas Corto Primero (SJF)

- Se asocia a cada proceso la longitud de su *próxima* ráfaga de CPU. Se usan estas longitudes para planificar los procesos con el tiempo mas corto.
- Dos esquemas:
 - **No apropiativo** – una vez que la CPU es asignada a un proceso, ésta no puede ser apropiada por otro proceso, hasta que el mismo complete su ráfaga de CPU.
 - **Apropiativo** – si llega un nuevo proceso con su ráfaga de CPU menor al tiempo de ejecución restante del proceso que está ejecutando, entonces se apropiada de la CPU. Este esquema es conocido como el Tiempo Remanente Mas Corto Primero (SRTF).
- SJF es óptimo – da el mínimo tiempo de espera promedio para un conjunto de procesos dado.

Ejemplo de SJF No Apropiativo

Proceso	Tiempo de llegada	Ráfaga
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

■ SJF (no apropiativo)

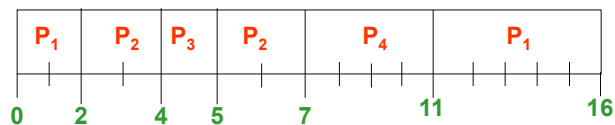


■ Tiempo medio de espera = $(0 + 6 + 3 + 7)/4 = 4$

Ejemplo SJF Apropiativo

Proceso	Tiempo de llegada	Ráfaga
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

■ SJF (apropiativo)



■ Tiempo medio de espera = $(9 + 1 + 0 + 2)/4 = 3$

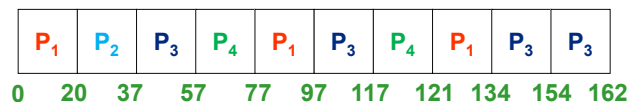
Planificación por Prioridad

- A cada proceso se le asocia un número (entero)
- El proceso con prioridad mas alta, es asignado en la CPU. (entero mas pequeño $\Rightarrow$ mas alta prioridad o el entero mas grande, *depende de la convención*).
 - Apropiativo
 - No apropiativo
- SJF es un caso especial del algoritmo de planificación por prioridad.
- **Problema** $\Rightarrow$ **Inanición** – los procesos de baja prioridad pueden no llegar a ejecutarse nunca.
- **Solución** $\equiv$ **Envejecimiento** – se incrementa la prioridad de los procesos en espera, a medida que pasa el tiempo, .

Ejemplo: RR con Quantum = 20

Proceso	Ráfaga
P_1	53
P_2	17
P_3	68
P_4	24

- La Diagrama de Gantt:

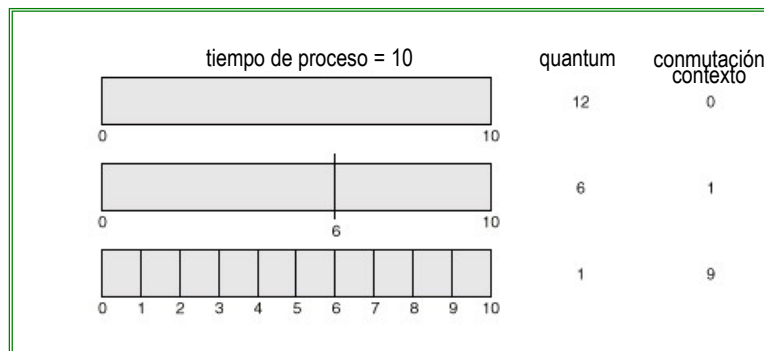


- Tipicamente, el tiempo de retorno promedio es mayor que el SJF, pero *mejor respuesta*.

Round Robin (RR)

- Cada proceso toma una pequeña unidad de tiempo de CPU (**quantum**), usualmente 10-100 milisegundos. Luego de este tiempo el proceso es quitado de la CPU y agregado a la cola de listos.
- Si hay n procesos en la cola de listos y el tiempo del quantum es q , entonces cada proceso toma $1/n$ del tiempo de CPU en rebanadas de a lo sumo q unidades de tiempo a la vez.
- Los procesos no esperan mas que $(n-1)q$ unidades de tiempo.
- El Rendimiento depende de q
 - q largo $\Rightarrow$ Primero-Entrar, Primero-Salir
 - q chico $\Rightarrow q$ debe ser grande con respecto al cambio de contexto, sino la sobrecarga es demasiado grande.

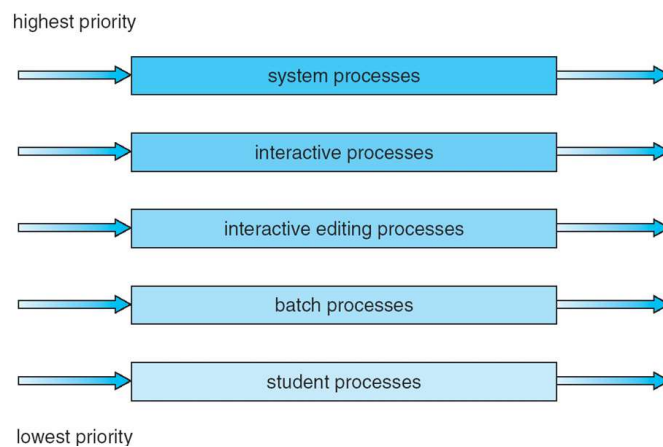
Un quantum PEQUEÑO Incrementa los Cambios de Contexto



Colas Multinivel

- La cola de listos está particionada en colas separadas:
 - foreground (interactive)
 - background (batch)
- Cada cola tiene su propio algoritmo de planificación,
 - foreground – RR
 - background – FCFS
- La planificación debe ser hecha entre las colas.
 - Planificación con prioridad fija; p.e., primero servir desde el foreground y luego del background. **Posible inanición.**
 - Tajada de tiempo – cada cola tiene una cierta cantidad de tiempo de CPU que puede planificar entre sus procesos;
x ejemplo: 80% del t de cpu para foreground en RR y el 20% del t de cpu para background en FCFS

Multilevel Queue Scheduling

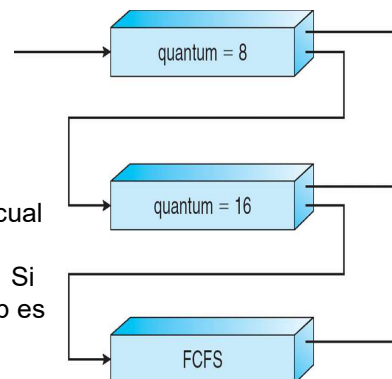


Colas Multinivel Realimentadas

- Un proceso puede moverse entre varias colas.
- El planificador de colas multinivel **realimentadas** está definido por los siguientes parámetros:
 - Número de colas
 - Algoritmos de planificación para cada cola
 - Método usado para determinar cuando mejorar un proceso (subir su prioridad)
 - Método usado para determinar cuando degradar un proceso (bajar su prioridad)
 - Método usado para determinar en que cola entra un proceso cuando necesita servicio.

Ejemplo de Colas Multinivel Realimentadas

- Tres colas:
 - Q_0 – quantum de 8 milisegundos
 - Q_1 – quantum de 16 milisegundos
 - Q_2 – FCFS
- Planificación
 - Un nuevo job entra a la cola Q_0 el cual es servido FCFS. Cuando gana la CPU, el job recibe 8 milisegundos. Si no finaliza en 8 milisegundos, el job es movido a la cola Q_1 .
 - En Q_1 el job es nuevamente servido FCFS y recibe 16 milisegundos adicionales. Si aún no completa, es movido a la cola Q_2 .



Planificación Multiprocesador

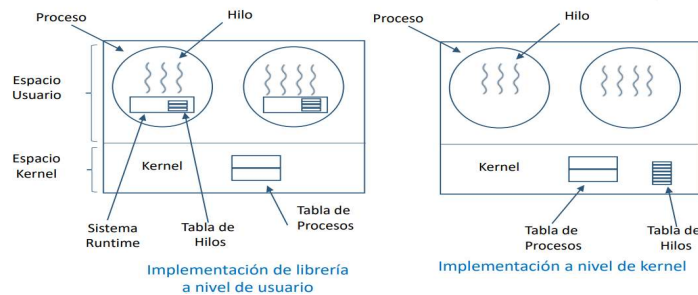
- Es mas compleja con múltiples CPUs.
- *Se puede compartir la carga, con procesadores homogéneos.*
- **Multiprocesamiento *Asimétrico***
 - (servidor maestro) solo un procesador accede a las estructuras de datos del sistema, simplificando la necesidad de compartir datos. Los demás procesadores ejecutan código de usuario.
- **Multiprocesamiento *Simétrico (SMP)*.**
 - Todos los procesos pueden estar en una cola común de procesos listos;
 - o cada procesador puede tener su propia cola privada de procesos listos.
 - Soportado por todos los SO modernos: a partir de Windows XP, Solaris, Linux y Mac OS X

Planificación Tiempo Real

- Tareas en tiempo real con cierto grado de *urgencia*
- Intentan controlar o reaccionar a eventos del mundo exterior que ocurren en tiempo real.
- **Sistemas de Tiempo Real Duro (estricto)** – deben **garantizar** que se completen las tareas de tiempo real *dentro de los plazos definidos*. (*prioridad con apropiación*). Los sistemas se utilizan para un único propósito, sobre dispositivos de tamaño pequeño y bajo costo.
- **Computación de Tiempo Real Blando** (no estricto)– requiere que los procesos críticos reciban prioridad de planificación **mayor** sobre otros procesos.

Planificación de Hilos

- En los modelos M:M y de 2 niveles se coloca una **estructura intermedia** (LWP) (Light-Weight Process)
- **Planificación Local** – Los hilos pertenecen al mismo proceso. Las librerías de hilos deciden poner el hilo en un LWP
- **Planificación Global** –el kernel decide que hilo del kernel (entre todos los hilos del Sistema) es el siguiente que corre



JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

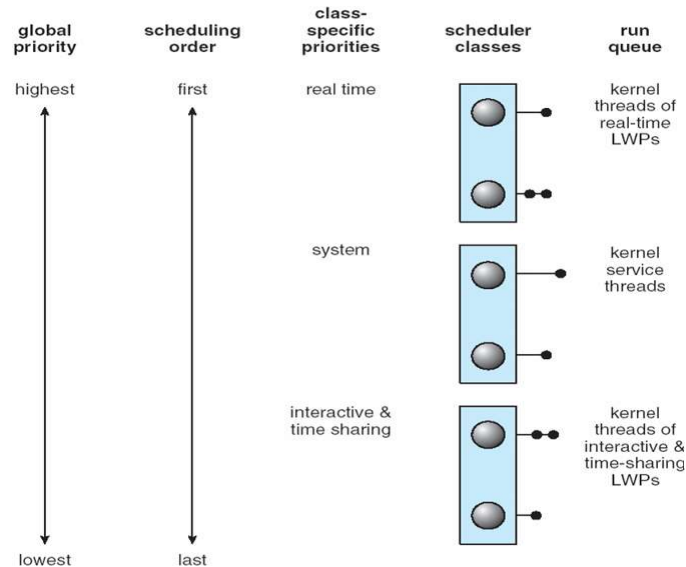
Ejemplos de Sistemas Operativos

- Planificación en Solaris
- Planificación en Windows XP
- Planificación en Linux

JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

Planificación en Solaris



JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

Tabla de Despacho en Solaris

Prioridad baja

priority	time quantum	quantum expirado	retorno del estado sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59

Prioridad alta

Nueva Prioridad asignada

JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

Prioridades en Windows XP

■ Clase de prioridades:

De tiempo real (16 – 31)

Variable (1-15)

R E L A T I V A		real-time	high	above normal	normal	below normal	idle priority
	time-critical	31	15	15	15	15	15
	highest	26	15	12	10	8	6
	above normal	25	14	11	9	7	5
	normal	24	13	10	8	6	4
	below normal	23	12	9	7	5	3
	lowest	22	11	8	6	4	2
	idle	16	1	1	1	1	1

- para gestión de memoria (Prioridad 0)

Prioridad base

Planificación en UNIX y Linux

La planificación tradicional en UNIX emplea **colas multinivel** (los niveles se definen en bandas de prioridades) usando RR en cada una de ellas:

$$P_j(i) = \text{Base}_j + \frac{\text{CPU}_j(i)}{2} + \text{nice}_j \quad (1) \quad \text{Es utilizada para ajustar dinámicamente la prioridad (por el uso de CPU).}$$

$$\text{CPU}_j(i) = \frac{\text{CPU}_j(i-1)}{2} \quad (2) \quad \text{implementa el "envejecimiento" cuando el proceso espera. Evita la inanición.}$$

$\text{CPU}_j(i)$ = Utilización del procesador por el proceso j en el intervalo i .

$P_j(i)$ = Prioridad del proceso j en el comienzo del intervalo i ;

valores bajos implican prioridades altas.

Base_j = Prioridad base del proceso j .

nice_j = Factor de ajuste controlable por el usuario

Planificación en UNIX y Linux

- En los primeros UNIX, la prioridad de cada proceso era computada cada segundo, hoy es cada *quantum*.
- El propósito de la prioridad base es dividir todos los procesos en bandas de niveles de prioridad.
- Los componentes *CPU* y *nice* se utilizan para prevenir que los procesos migren fuera de su banda asignada (dada por la prioridad base).
- Las bandas son utilizadas para optimizar el acceso a los dispositivos que se manejan con bloques de información (discos, cintas, CD, etc) y permitir al SO responder rápidamente a las llamadas al sistema.

Planificación en UNIX y Linux

En orden decreciente de prioridad, las bandas son:

- ▶ Swapper.
- ▶ Control de dispositivos de E/S en bloques.
- ▶ Manipulación de archivos.
- ▶ Control de dispositivos de E/S por caracteres.
- ▶ Procesos de usuarios.



Dentro de esta banda, el uso de la historia de ejecución tiende a penalizar a los procesos limitados por CPU a costa de los procesos limitados por E/S.

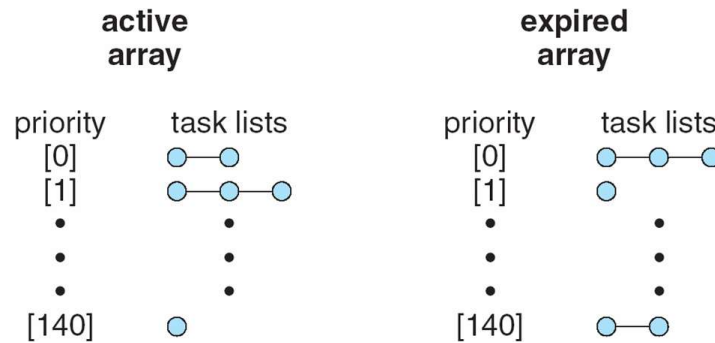
Planificación en Linux

- Dos algoritmos: tiempo compartido (100-140) y tiempo real (0-99)
- Tiempo compartido
 - Por Prioridad basado en créditos – son planificados primero los procesos con más créditos
 - Los créditos se restan cuando ocurre la interrupción del timer
 - Cuando el crédito = 0, es elegido otro proceso
 - Cuando todos los procesos tienen crédito = 0, ocurre una reacreditación
 - ▶ Basada en factores que incluyen *prioridad* e *historia*
- Tiempo real
 - Tiempo real blando
 - Compatible Posix– dos clases
 - ▶ FCFS y RR
 - ▶ Siempre corre primero el Proceso de más alta prioridad

Relación entre Prioridades y Longitud del quantum

numeric priority	relative priority		time quantum
0	highest	real-time tasks	200 ms
•			
•			
•			
99			
100		other tasks	
•			
•			
•			
140	lowest		10 ms

Lista de Tareas Indexadas de Acuerdo a Prioridades



JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

Evaluación de Algoritmos

- **Modelo Determinístico** – toma una carga de trabajo predeterminada concreta y define el rendimiento de cada algoritmo para esa carga.
- **Modelo de colas** – describe el sistema informático como una red de servidores. C/u con una cola de procesos en espera (CPU, disp. de E/S). Usa el análisis matemático y algoritmos complejos que resultan difíciles y poco realistas. Estos modelos solo representan una aproximación a los sistemas reales.
- **SIMULACIONES** - requiere programar un modelo, con una precisión limitada. La generación de datos puede ser Aleatorio o basados en registros reales.
- Para evaluar de forma precisa un algoritmo hay que codificar junto al SO y verlo funcionar.
- Los algoritmos de planificación flexibles pueden ser modificados por los administradores de sistema y ajustarse a aplicaciones específicas.
- Otro método es emplear APIs permitiendo modificar la prioridad de un proceso. API de Java, POSIX y Windows

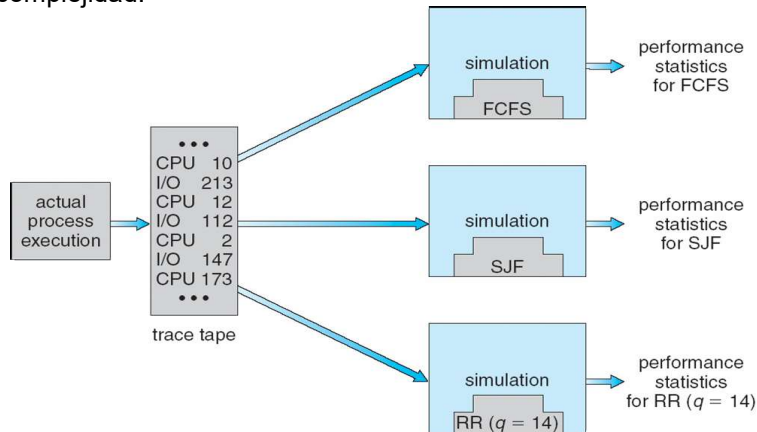
JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

Evaluación de Algoritmos

La simulación, requieren muchas horas de t. de computadora y gran cantidad de espacio de almacenamiento.

El diseño, codificación y depuración del simulador puede ser de gran complejidad.



JRA © - LRS 2025

Sistemas Operativos –Planificación de Procesos

Fin

Módulo 5

Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia "San Juan Bosco"